Intent Compression Architecture

A Control Plane for Intent in Reliable LLM Systems

Engineering design proposal

Author	Paul Maddison
Short name	ICA
Canonical artifacts	README.md, architecture diagram, proposal DOCX/PDF, evaluation scaffold

Executive summary. ICA is a pre-generation control layer and control plane for intent. It decides whether clarification is worth the cost before the model commits to an answer or an agent action. It treats ambiguity as uncertainty over latent user intent, estimates expected utility over possible clarification replies, and routes the request to direct answer, clarifier, premise-check, or refusal/redirect as appropriate.

1. Architecture overview

A common failure mode in LLM systems is unresolved ambiguity. Users often ask questions that admit multiple plausible interpretations, while the system is optimized to answer immediately. The result is a broad first answer, a user correction, and an unnecessary second pass.

ICA inserts a control layer between user input and final generation. The layer estimates likely intent hypotheses, scores ambiguity and risk, and decides whether clarification improves the expected outcome enough to justify the extra turn.

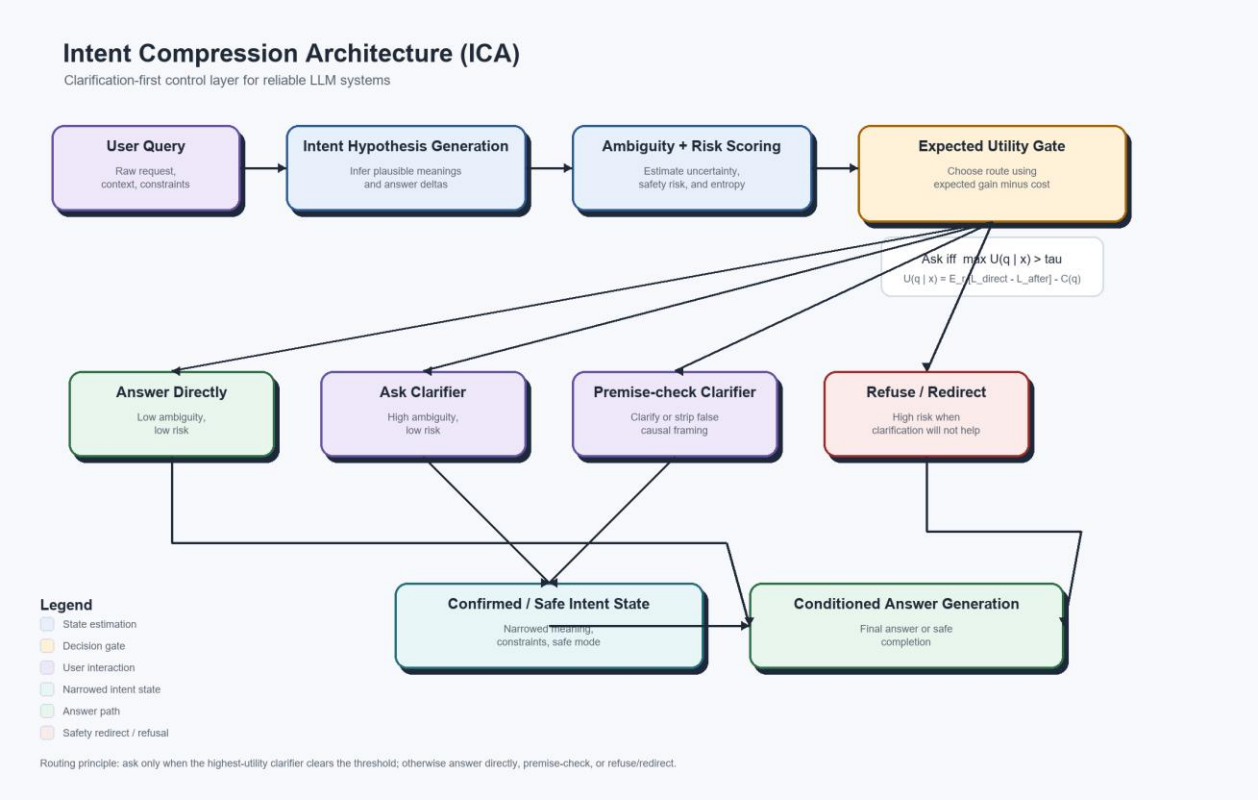

Figure 1. ICA control flow with expected-utility routing and embedded legend.

2. Core decision policy

Minimal rule: ask a clarifying question only when expected improvement in answer quality or safety exceeds the cost of another turn.

$$U(q \mid x) = E_r\,[L_{direct}(x) - L_{after}(x, q, r)] - C(q)$$

$$\text{Ask iff } \max_q U(q \mid x) > \tau_{domain}$$

This formulation makes the unknown user reply explicit by taking an expectation over possible clarification responses. The domain threshold allows the same architecture to be tuned differently for coding, medical, legal, finance, or customer-support use cases.

3. Estimation and calibration

The schema fields intent_entropy_bits and intent_hypotheses[].probability are not meant to be accepted as magical model self-knowledge. In production they should be calibrated control-layer estimates: generate mutually exclusive answer-changing intent hypotheses, include an other bucket, estimate probabilities from a calibrated classifier or semantic clusters of repeated samples, and then compute entropy mechanically as -sum p log2 p.

Candidate utility should be treated the same way. The repository policy module uses provider-estimated clarifier benefit as one input, then applies deterministic local adjustments for token cost, latency cost, turn friction, and optional risk adjustment. That keeps the ask-vs-answer decision auditable even when upstream estimates are noisy.

Tau should be tuned, not guessed. The simplest calibration is a grid search on held-out labeled prompts that minimizes the combined loss from unnecessary clarification, false direct answers, false refusals, token cost, latency, abandonment, and safety risk. Operational dashboards should watch both failure modes: over-clarification when tau is too low and silent wrong-funnel answers when tau is too high.

4. Compression interpretation

ICA treats ambiguity as entropy over possible user intents. Before clarification, the system reasons over H(I | x). After clarification q and user reply r, the system reasons over H(I | x, q, r). The expected value of clarification is the reduction in intent entropy, tempered by interaction cost.

This is why the term intent compression is mathematically defensible: the controller narrows the intent distribution before generation rather than letting the answering model hedge across multiple meanings.

5. Routing policy

Ambiguity	Risk	Default action	Operational note
Low	Low	Answer directly	No clarification needed.

High	Low	Ask one high-value clarifier	Clarify only if the best question clears the threshold.
Low	High	Direct safe completion, premise-check, or refuse/redirect	Do not ask the user to reconfirm harmful intent if the safe response would not change.
High	High	Strict clarification, constrained answer, or refuse/redirect	High uncertainty plus high downside requires tighter control.

A practical implication is that clarification is not always the right response to risk. Sometimes the cleanest and safest route is a direct refusal with a helpful redirect.

6. Prior-art positioning

ICA is closest to selective-clarification and ambiguous-question-answering work such as CLAM, CLAMBER, and AmbigQA. It also relates to ReAct-style agent loops and uncertainty methods such as self-consistency and semantic entropy.

The contribution claimed here is narrower and more architectural: clarification is specified as a pre-generation routing contract with explicit ambiguity, risk, utility, threshold, route, and counter-metric fields. The novelty is not asking questions; it is engineering ask-vs-answer as a calibrated control layer.

Primary references: CLAM (arxiv.org/abs/2212.07769), CLAMBER (arxiv.org/abs/2405.12063), AmbigQA (arxiv.org/abs/2004.10645), ReAct (arxiv.org/abs/2210.03629), Self-Consistency (arxiv.org/abs/2203.11171), and Semantic Uncertainty (arxiv.org/abs/2302.09664).

7. Implementation contract

The control layer should emit a structured decision object rather than a free-form paragraph. This proposal includes a JSON schema and reference example in spec/clarifier_output.schema.json and spec/clarifier_output.example.json.

The repository also includes a lightweight schema validation path so the example can be tested against the contract rather than presented as a static mockup. Local validation is the canonical reproducibility path.

```
{
  "ambiguity_score": 0.74,
  "risk_score": 0.22,
  "decision": "ask_clarifier",
  "clarifying_question": "Do you mean propaganda as in biased or one-sided political
messaging intended to influence opinion?",
  "answer_constraints": ["avoid loaded framing", "define terms before conclusion"]
}
```

8. Hidden baseline cost

A short first answer is not necessarily an efficient answer. In ambiguous prompts, a baseline system can choose the wrong interpretation, give a broad or misleading answer, and force the user into a correction funnel before the real issue is finally exposed.

This is not only a retry problem. It is a wrong-funnel problem. The user either spends turns arguing the model into discovering the ambiguous term, or leaves with a partially wrong answer without ever learning what caused the mismatch.

For that reason, ICA should be evaluated on tokens to resolved intent, definition-discovery turn, correction-funnel depth, and user correction burden, not just on the cost of the first assistant response.

The human-behavior risk is that the repair funnel often never happens. A user may leave after the first answer, accept the wrong semantic frame, or screenshot the first answer as evidence for a contested claim. The Elon Musk propaganda example is a compact case: the answer changes depending on whether propaganda means biased or one-sided political messaging intended to influence opinion, coordinated deceptive messaging, or deliberate misinformation. The compression-optimized clarifier is a short yes/no gate: Do you mean propaganda as in biased or one-sided political messaging intended to influence opinion?

ICA should therefore track early-exit silent-failure risk and screenshot misuse risk alongside ordinary retry metrics. The goal is to prevent a screenshotable first answer from becoming social proof for a meaning the user and model never agreed on.

9. Evaluation package

The repository now includes a benchmark prompt set, an evaluation protocol, a sample reporting format, and a first-pass pilot benchmark. The evaluation compares not only direct one-shot answers, but also direct answers followed by the repair funnel when the first answer misses the intended meaning. The next credibility jump is a multi-rater or live-user benchmark.

The published pilot is designed to test ambiguous-prompt handling, not to estimate production-wide clarification frequency.

Artifact	Purpose
examples/ambiguous_prompts.csv	Starter prompt set covering low-risk ambiguity, high-risk ambiguity, and premise-risk cases.
eval/README.md	Protocol, scoring rubric, success criteria, and counter-metrics.
eval/sample_results.md	Template for benchmark reporting without fabricating results.
eval/pilot_results.md	Human-readable first-pass pilot benchmark report.
eval/pilot_results.csv	Machine-readable pilot results table for analysis and reuse.

Key primary metrics include first assistant-message tokens, total tokens to resolved intent, definition-discovery turn, retry count, correctness, clarity, and premise handling. Key counter-metrics include over-clarification rate, false direct-answer rate, silent-failure proxy, false refusal rate, and clarification bias.

The benchmark now also distinguishes early-exit silent-failure risk from screenshot misuse risk. Early-exit risk means the user plausibly leaves before the ambiguous term is exposed. Screenshot misuse risk means the first answer can be quote-mined as evidence for a contested, misleading, or unsafe interpretation.

The pilot utility proxy is transparent rather than implicit: quality = (correctness + clarity + safety) / 3; utility_proxy = quality - 0.01 * total_tokens - 0.5 * retries. This proxy rewards judged final-answer quality and penalizes token cost and retry burden. It does not measure live satisfaction, abandonment, wall-clock latency, or revenue impact.

The current 25-prompt pilot is deliberately ambiguity-heavy and routes 20 of 25 prompts to ask_clarifier. That 80 percent clarification rate is defensible for a stress set, but it is not a desired production rate. On representative traffic, clarification rate, over-clarification rate, unnecessary clarification rate, and false direct-answer rate should become the primary tau-calibration signals.

The circularity risk is real. The current pilot is single-rater and uses evaluator-supplied clarification replies. The next benchmark should separate reply generation from scoring, use route-blind or independent review where feasible, report inter-rater agreement, and tune tau on a separate split from the headline evaluation.

10. Strategic implication

At large scale, ICA is not only a reliability pattern. It becomes a clarification data flywheel. Each ambiguous query, clarifier, user reply, and final outcome produces a structured intent-resolution

trace that is more valuable than an ordinary chat log because it captures what the user actually meant.

That data advantage only exists if the system logs and acts on the traces: capture candidate intents, probabilities, rejected clarifiers, tau, selected route, user reply, and final outcome; label resolved intent with privacy controls; recalibrate probabilities and thresholds per domain; retrain the policy on both good clarifiers and cases where the system should not have asked; and deploy changes through A/B tests that watch success metrics and counter-metrics.

Public OpenAI disclosures in 2026 describe ChatGPT as having more than 900 million weekly active users and more than 50 million consumer subscribers. At that scale, even rare ambiguity classes become common in absolute terms. A provider with that distribution can improve ambiguity detection, ask-vs-answer thresholds, neutral clarifier wording, risk handling, and future base-model behavior using real-world traces rather than only synthetic or expert-labeled data.

The architecture is copyable. The live, high-volume intent-resolution feedback loop is much harder to copy. That is why ICA can be understood not only as a UX improvement, but as a route by which market share becomes model-quality advantage.

11. Future extension: coding agents

The same control-layer idea may apply to coding agents. In chat, ICA prevents the model from answering before intent is resolved. In agents, ICA can preserve the original task intent while filtering tool-output noise, failed attempts, and conversational drift across many steps.

In that framing, ICA becomes a control plane for agentic systems. It does not replace the model's reasoning or tool use; it governs the state the model should reason over: original goal, verified facts, current constraints, failed paths, and the next useful action.

For an agent, the compressed state packet might include the original user goal, current verified state, known constraints, failed attempts to avoid, and next best action. That packet can keep the model focused on the task state that matters rather than the full conversational residue.

This repository does not yet benchmark coding-agent performance. The extension is a natural, testable bridge from clarification-first chat to intent-preserving agent orchestration, with metrics such as tokens to green tests, repeated mistakes, time to resolution, final pass rate, unnecessary code churn, and preservation of original intent. If validated on coding-agent benchmarks, this could improve agent unit economics by reducing repeated context, failed loops, and intent drift.

12. Adversarial and deployment guidance

ICA is most useful when deployed as a policy layer around an existing model stack. The orchestration logic should be explicit and testable, while uncertainty is isolated to model outputs and external-system calls such as retrieval, APIs, or mutable external state.

The controller should log ambiguity score, risk score, candidate clarifiers, expected utility estimates, final route, and user reply when applicable. Those traces are the data needed to improve the control layer over time.

A clarification layer is also a control surface. It can be probed by users who try to trigger or suppress clarification, launder harmful intent through ambiguity, steer unsafe options through clarifier replies, inject desired routing labels, overload the controller with many plausible intents, or poison future traces. Mitigations include internal thresholds, independent risk scoring, post-reply safety re-scoring, policy-controlled hypothesis generation, capped hypothesis sets with an other bucket, and separation between online traces and trusted training labels.

13. Conclusion

ICA is a credible architecture pattern because it defines a control-layer problem that engineers can implement: infer intent hypotheses, quantify ambiguity, route by expected utility, narrow intent before generation when doing so is worth the cost, and build systems that are more precise, more reliable, easier to defend, and often cheaper to operate once wrong-funnel conversations are counted properly.

Appendix A. Starter benchmark pack

The repository includes a lightweight evaluation package so the proposal can move from theory to early evidence without requiring a large research program.

Artifact	Use
examples/ambiguous_prompts.csv	Seed prompt set covering coding, planning, legal, medical, finance, public reasoning, and safety-sensitive ambiguity.
eval/README.md	Evaluation procedure, rating rubric, success criteria, and counter-metrics.
eval/sample_results.md	Reporting format for summary metrics and per-prompt comparisons.
eval/pilot_results.md	Published first-pass pilot benchmark report.
eval/pilot_results.csv	Structured pilot dataset for re-analysis or extension.

Pilot signal snapshot

In the current 25-prompt pilot, direct first-pass answers required a repair funnel in 19 cases. On that same prompt set, ICA reduced mean definition-discovery turn from 2.6 to 1.0 and reduced mean total tokens to satisfactory resolution from 91.68 on the repaired baseline path to 78.04.

That is the narrower but stronger efficiency claim: early clarification is not always shorter than a one-shot answer, but it is often cheaper than discovering the same ambiguity after the system has already committed to the wrong semantic funnel.

In repaired-baseline scoring, final answer quality is equalized with ICA only when the baseline needed repair. The repaired utility score still penalizes extra repair tokens and retry burden so delayed clarification does not receive a free tie.

Reproduction path: install dependencies from requirements.txt, or requirements.lock for a pinned replay, run the local validation helper script, inspect the generated CSV and Markdown report, then regenerate the proposal artifacts if needed.

Recommended next move: extend the single-rater pilot into a multi-rater or API-instrumented benchmark, then update the controller threshold and routing rules based on observed over-clarification, correction-funnel depth, silent-failure risk, false direct-answer, and false-refusal rates.